

Chapter 8 – Software Testing

COMPLETE EXAM PREPARATION NOTES

True/False | Short Questions (2 / 3 / 5 Marks)

PART A — KEY CONCEPTS QUICK REFERENCE

1. What is Software Testing?

- Testing is intended to show that a program does what it is intended to do and to discover program defects before it is put into use.
- You execute a program using artificial data and check results for errors, anomalies, or non-functional attributes.
- Testing can reveal the PRESENCE of errors — NOT their ABSENCE.
- Testing is part of a more general Verification and Validation (V&V) process, which also includes static validation techniques.

2. Two Goals of Testing

- Validation Testing: To demonstrate to the developer and customer that the software meets its requirements. A successful test shows the system operates as intended.
- Defect Testing: To discover faults/defects where behavior is incorrect or not in conformance with specification. A successful defect test makes the system perform incorrectly, exposing a defect.

3. Stages of Testing

- Development Testing — testing during development to discover bugs and defects.
- Release Testing — a separate team tests a complete version before it is released to users.
- User Testing — users/potential users test the system in their own environment.

4. Development Testing (3 Levels)

- Unit Testing: Testing individual program units or object classes in isolation. Focus: functionality of objects/methods.
- Component Testing: Several units integrated to create composite components. Focus: component interfaces.
- System Testing: Components integrated; tested as a whole. Focus: component interactions and emergent behavior.

5. Unit Testing

- Unit testing is a defect testing process.
- Units may be: individual functions/methods, object classes with several attributes/methods, or composite components with defined interfaces.

- Complete test coverage of a class = testing ALL operations, setting/interrogating ALL attributes, exercising object in ALL possible states.
- Inheritance makes object class testing more difficult as information is not localised.

6. Automated Testing

- Unit testing should be automated wherever possible — tests run and checked without manual intervention.
- Uses a test automation framework such as JUnit.
- Frameworks provide generic test classes; results are reported through a GUI.

7. Testing Strategies

- Partition Testing: Identify groups of inputs with common characteristics that should be processed in the same way (equivalence partitions/domains). Choose tests from within each partition.
- Guideline-based Testing: Use testing guidelines reflecting previous experience of common programmer errors.

8. Testing Guidelines for Sequences

- Test with sequences having only a single value.
- Use sequences of different sizes in different tests.
- Derive tests so first, middle, and last elements of the sequence are accessed.
- Test with sequences of zero length.

9. General Testing Guidelines

- Choose inputs that force the system to generate all error messages.
- Design inputs that cause input buffers to overflow.
- Repeat the same input or series of inputs numerous times.
- Force invalid outputs to be generated.
- Force computation results to be too large or too small.

10. Component Testing & Interface Types

- Software components are composite components made up of several interacting objects.
- Testing composite components focuses on showing that the component interface behaves according to its specification.
- Interface Types:
 - ► Parameter interfaces — Data passed from one method/procedure to another.
 - ► Shared memory interfaces — Block of memory shared between procedures/functions.
 - ► Procedural interfaces — Sub-system encapsulates a set of procedures to be called by other sub-systems.
 - ► Message passing interfaces — Sub-systems request services from other sub-systems.

11. Interface Errors

- Interface misuse — Calling component makes an error in use of the interface (e.g., wrong parameter order).

- Interface misunderstanding — Calling component embeds incorrect assumptions about the called component's behaviour.
- Timing errors — Called and calling components operate at different speeds; out-of-date information is accessed.

12. Interface Testing Guidelines

- Design tests so parameters to called procedure are at extreme ends of their ranges.
- Always test pointer parameters with null pointers.
- Design tests which cause the component to fail.
- Use stress testing in message passing systems.
- In shared memory systems, vary the order in which components are activated.

13. System Testing

- Focuses on testing interactions between components.
- Checks components are compatible, interact correctly, and transfer the right data at the right time.
- Tests the emergent behaviour of a system.
- System testing is a collective rather than an individual process.
- May involve a separate testing team with no involvement from designers and programmers.

14. Testing Policies

- Exhaustive system testing is impossible, so testing policies define required system test coverage.
- Examples: All system functions accessed through menus should be tested; combinations of functions accessed through same menu must be tested; all functions must be tested with both correct and incorrect user input.

15. Test-Driven Development (TDD)

- TDD is an approach where testing and code development are interleaved.
- Tests are written BEFORE code — 'passing' the tests is the critical driver of development.
- Code is developed incrementally, along with a test for each increment.
- You do NOT move to the next increment until current code passes its test.
- TDD was introduced as part of agile methods (e.g., Extreme Programming). Can also be used in plan-driven development.

16. TDD Process Activities

1. Identify the small increment of functionality required.
2. Write an automated test for this functionality.
3. Run the test — initially the new test WILL FAIL (functionality not yet implemented).
4. Implement the functionality and re-run the test.
5. Once all tests pass, move on to the next chunk of functionality.

17. Benefits of TDD

- Code Coverage — Every code segment written has at least one associated test.
- Regression Testing — A regression test suite is developed incrementally as the program develops.

- Simplified Debugging — When a test fails, the newly written code is checked and modified; the problem location is obvious.
- System Documentation — Tests are a form of documentation that describe what the code should do.

18. Regression Testing

- Testing the system to check that changes have not 'broken' previously working code.
- In manual testing, regression testing is expensive; with automated testing, it is simple and straightforward.
- All tests are rerun every time a change is made to the program.
- Tests must run successfully BEFORE the change is committed.

19. Release Testing

- Process of testing a particular release of a system intended for use outside of the development team.
- Primary goal: convince the supplier (and customer) that the system is good enough for use.
- Must show: specified functionality, performance, dependability, and no failure during normal use.
- Usually a BLACK-BOX testing process — tests derived only from the system specification.

20. Release Testing vs. System Testing

- A SEPARATE team (not involved in development) is responsible for release testing.
- System testing (by dev team) focuses on discovering bugs (defect testing).
- Release testing checks that system meets requirements and is good enough for external use (validation testing).

21. User Testing — Types

- Alpha Testing — Users work with the development team to test software AT THE DEVELOPER'S SITE.
- Beta Testing — A release is made available to users to experiment and raise problems with system developers.
- Acceptance Testing — Customers test a system to decide whether it is ready to be accepted and deployed. Primarily for CUSTOM systems.
- User testing is essential even after comprehensive system and release testing because influences from the user's working environment (reliability, performance, usability, robustness) cannot be replicated in a testing environment.

PART B — TRUE / FALSE QUESTIONS

Study every statement carefully. Green = TRUE, Red = FALSE.

Topic 1: Program Testing — Basics

Statement	T / F
Testing can reveal the presence of errors but NOT their absence.	TRUE
Testing is intended to show that a program does what it is intended to do.	TRUE
When testing software, you execute a program using real production data.	FALSE
Testing can prove that a program is completely free of errors.	FALSE
Testing is part of a more general verification and validation process.	TRUE
Static validation techniques are not part of the V&V process.	FALSE

Topic 2: Goals of Testing

Statement	T / F
Validation testing aims to demonstrate that the software meets its requirements.	TRUE
Defect testing uses test cases that reflect how the system is normally used.	FALSE
For custom software, there should be at least one test for every requirement.	TRUE
A successful defect test is one that shows the system operates correctly.	FALSE
A successful validation test shows that the system operates as intended.	TRUE
Defect testing cases can be deliberately obscure.	TRUE

Topic 3: Stages of Testing

Statement	T / F
There are three main stages of testing: development, release, and user testing.	TRUE
Release testing is performed by the same team that developed the system.	FALSE
User testing is carried out by users in their own environment.	TRUE
Development testing is done only after the system is fully complete.	FALSE

Topic 4: Development Testing

Statement	T / F
Development testing includes unit, component, and system testing.	TRUE
Unit testing focuses on testing component interfaces.	FALSE
Component testing focuses on testing component interfaces.	TRUE
System testing focuses on testing component interactions.	TRUE
Unit testing is a defect testing process.	TRUE

System testing tests the emergent behaviour of a system.	TRUE
--	------

Topic 5: Unit Testing

Statement	T / F
Unit testing is the process of testing individual components in isolation.	TRUE
Object class testing requires testing all operations, all attributes, and all states.	TRUE
Inheritance makes object class testing easier because information is localized.	FALSE
Composite components with defined interfaces can be units in unit testing.	TRUE

Topic 6: Automated Testing

Statement	T / F
Unit testing should be automated wherever possible.	TRUE
JUnit is an example of a test automation framework.	TRUE
Automated unit testing requires manual checking after every test run.	FALSE
Test automation frameworks provide generic test classes that you extend.	TRUE

Topic 7: Testing Strategies

Statement	T / F
In partition testing, you should choose tests from within each equivalence partition.	TRUE
Each equivalence partition is a class where the program behaves equivalently for each member.	TRUE
Guideline-based testing is based on common programmer error patterns.	TRUE
You must test every possible input value in partition testing.	FALSE

Topic 8: Interface Testing

Statement	T / F
There are four main types of interfaces: parameter, shared memory, procedural, and message passing.	TRUE
Interface misuse occurs when a calling component embeds incorrect assumptions about the called component.	FALSE
Interface misunderstanding occurs when a calling component embeds incorrect assumptions.	TRUE
Timing errors occur when called and calling components operate at different speeds.	TRUE
Null pointers should always be used to test pointer parameters.	TRUE
Stress testing is used in shared memory systems to detect timing errors.	FALSE
Stress testing is used in message passing systems.	TRUE

Topic 9: System Testing

Statement	T / F
System testing checks that components are compatible and interact correctly.	TRUE
System testing is an individual rather than a collective process.	FALSE
Exhaustive system testing is possible with enough time and resources.	FALSE
Some companies use a separate testing team with no involvement from programmers.	TRUE
System testing focuses on testing interactions between components.	TRUE

Topic 10: Test-Driven Development (TDD)

Statement	T / F
In TDD, code is written before tests.	FALSE
In TDD, tests are written before code.	TRUE
TDD was introduced as part of agile methods such as Extreme Programming.	TRUE
In TDD, you can move to the next increment before the current test passes.	FALSE
TDD can only be used in agile (not plan-driven) development processes.	FALSE
When a new test is first run in TDD, it will initially fail.	TRUE
The tests in TDD serve as a form of system documentation.	TRUE

Topic 11: Regression Testing

Statement	T / F
Regression testing checks that changes have not broken previously working code.	TRUE
With automated testing, regression testing is expensive and time-consuming.	FALSE
All tests must be rerun every time a change is made in automated regression testing.	TRUE
Changes can be committed before tests run successfully.	FALSE

Topic 12: Release Testing

Statement	T / F
Release testing is usually a black-box testing process.	TRUE
The primary goal of release testing is to impress the development team.	FALSE
Release testing tests are derived from the system specification.	TRUE
Release testing is performed by the development team.	FALSE
Release testing is a form of system testing.	TRUE
System testing by developers focuses on validation testing.	FALSE

Release testing focuses on validation testing.	TRUE
--	------

Topic 13: User Testing

Statement	T / F
Alpha testing is done at the developer's site with users and the development team.	TRUE
Beta testing involves releasing software to users to find problems.	TRUE
Acceptance testing is primarily for generic off-the-shelf software.	FALSE
Acceptance testing is primarily for custom systems.	TRUE
User testing is unnecessary if comprehensive system and release testing have been done.	FALSE
The user's working environment can be perfectly replicated in a testing environment.	FALSE

PART C — SHORT QUESTIONS (2 / 3 / 5 MARKS)

☆ 2-Mark Questions

Q: What is software testing? [2 marks]

Ans: Software testing is the process of executing a program using artificial data to show that the program does what it is intended to do and to discover defects before it is put into use. Testing can reveal the presence of errors but NOT their absence.

Q: What is the difference between validation testing and defect testing? [2 marks]

Ans: Validation testing demonstrates that the software meets its requirements — a successful test shows the system works as intended. Defect testing discovers faults where behavior is incorrect — a successful test makes the system perform incorrectly, exposing a defect.

Q: What is unit testing? [2 marks]

Ans: Unit testing is the process of testing individual components (functions, methods, object classes, or composite components) in isolation. It is a defect testing process focused on the functionality of objects or methods.

Q: What is regression testing? [2 marks]

Ans: Regression testing is the process of testing a system to check that changes have not 'broken' previously working code. With automated testing, all tests are rerun every time a change is made; tests must pass before any change is committed.

Q: What is Test-Driven Development (TDD)? [2 marks]

Ans: TDD is an approach where testing and code development are interleaved. Tests are written BEFORE code, and passing the tests is the critical driver of development. Code is developed incrementally with a test for each increment.

Q: What is release testing? [2 marks]

Ans: Release testing is the process of testing a particular release of a system intended for use outside the development team. Its primary goal is to convince the supplier that the system is good enough for use. It is usually a black-box testing process.

Q: What is equivalence partitioning? [2 marks]

Ans: Equivalence partitioning is a testing strategy where input data is grouped into partitions (equivalence domains) in which the program behaves equivalently for each member. Test cases are chosen from within each partition to reduce the total number of tests needed.

Q: Name the three types of user testing. [2 marks]

Ans: 1. Alpha testing — users work with the development team at the developer's site. 2. Beta testing — a release is made available to users to experiment and find problems. 3. Acceptance testing — customers test the system to decide if it is ready for deployment (for custom systems).

Q: What is automated unit testing? [2 marks]

Ans: Automated unit testing uses a test automation framework (e.g., JUnit) to write and run program tests without manual intervention. The framework provides generic test classes that are extended to create specific test cases, and results are reported through a GUI.

Q: Name the four types of component interfaces. [2 marks]

Ans: 1. Parameter interfaces — data passed between methods/procedures. 2. Shared memory interfaces — block of memory shared between procedures. 3. Procedural interfaces — sub-system encapsulates procedures called by other sub-systems. 4. Message passing interfaces — sub-systems request services from each other.

☆☆ 3-Mark Questions

Q: What are the three stages of testing? Explain each briefly. [3 marks]

Ans: 1. Development Testing: Testing activities carried out by the development team during system development to discover bugs and defects. Includes unit, component, and system testing. 2. Release Testing: A separate team tests a complete version of the system before it is released to users. Focus is on validation — checking the system meets requirements. 3. User Testing: Users or potential users test the system in their own environment. Essential because the user's working environment has major effects on reliability, performance, and usability that cannot be replicated in a testing environment.

Q: What are the three levels of development testing? What does each focus on? [3 marks]

Ans: 1. Unit Testing — tests individual program units or object classes in isolation; focuses on the functionality of objects or methods. 2. Component Testing — integrates several units into composite components; focuses on testing component interfaces. 3. System Testing — integrates all/some components; focuses on testing component interactions and emergent system behavior.

Q: What are the three types of interface errors? Explain each. [3 marks]

Ans: 1. Interface Misuse — A calling component calls another component but makes an error in its use of the interface (e.g., passing parameters in the wrong order). 2. Interface Misunderstanding — A calling component embeds incorrect assumptions about the behavior of the called component. 3. Timing Errors — The called and calling components operate at different speeds and out-of-date information is accessed.

Q: What are the TDD process activities? [3 marks]

Ans: 1. Identify a small increment of functionality to implement. 2. Write an automated test for that functionality. 3. Run the test — it will initially FAIL since the functionality is not yet implemented. 4. Implement the functionality and re-run the test. 5. Once all tests pass, move on to the next increment of functionality.

Q: What are the benefits of Test-Driven Development? [3 marks]

Ans: 1. Code Coverage — every code segment written has at least one associated test. 2. Regression Testing — a test suite is developed incrementally as the program is developed. 3. Simplified Debugging — when a test fails, the problem is obvious (the newly written code); it is easily checked and modified. 4. System Documentation — the tests describe what the code should be doing, serving as a form of documentation.

Q: How does release testing differ from system testing? [3 marks]

Ans: 1. Team: Release testing is done by a separate team NOT involved in development; system testing is done by the development team. 2. Purpose: System testing (by dev team) focuses on discovering bugs (defect testing); release testing checks the system meets requirements and is good enough for external use (validation testing). 3. Both are forms of system testing, but release testing is a black-box process derived from the system specification.

Q: What are interface testing guidelines? [3 marks]

Ans: 1. Design tests so parameters to called procedures are at extreme ends of their ranges. 2. Always test pointer parameters with null pointers. 3. Design tests that cause the component to fail. 4. Use stress testing in message passing systems. 5. In shared memory systems, vary the order in which components are activated.

Q: What is complete test coverage of a class in object class testing? [3 marks]

Ans: Complete test coverage of a class involves: 1. Testing ALL operations associated with an object. 2. Setting and interrogating ALL object attributes. 3. Exercising the object in ALL possible states. Note: Inheritance makes object class testing more difficult as information to be tested is not localised.

☆☆☆ 5-Mark Questions

Q: Explain the two main goals of software testing with examples. [5 marks]

Ans: Goal 1 — Validation Testing: This leads to validation testing where you expect the system to perform correctly using test cases that reflect the system's expected use. For custom software, there should be at least one test for every requirement in the requirements document. For generic software, there should be tests for all system features and combinations of features. A successful validation test shows the system operates as intended. Goal 2 — Defect Testing: This leads to defect testing where the aim is to discover situations in which behavior is incorrect, undesirable, or does not conform to specification. The test cases are designed to expose defects — they can be deliberately obscure and need not reflect normal use. Defect testing is concerned with rooting out system crashes, unwanted interactions, incorrect computations, and data corruption. A successful defect test makes the system perform incorrectly, exposing a defect.

Q: What is Test-Driven Development? Explain its process and benefits. [5 marks]

Ans: Definition: TDD is an approach to program development where testing and code development are interleaved. Tests are written BEFORE code, and passing the tests drives development. Code is developed incrementally with a test for each increment. TDD was introduced as part of agile methods (e.g., Extreme Programming) but can also be used in plan-driven processes. Process: 1. Identify a small increment of functionality required. 2. Write an automated test for this functionality. 3. Run the test — it initially FAILS because the functionality is not yet implemented. 4. Implement the functionality and re-run the test. 5. Once all tests pass, move on to the next increment. Benefits: 1. Code Coverage — every code segment has at least one test. 2. Regression Testing — a test suite is built incrementally. 3. Simplified Debugging — the newly written code is the obvious place to look when a test fails. 4. System Documentation — tests describe what the code should do.

Q: Discuss the three types of user testing and why user testing is essential. [5 marks]

Ans: Types of User Testing: 1. Alpha Testing — Users work with the development team to test the software at the DEVELOPER's site. The team can observe and assist users directly. 2. Beta Testing — A release of the software is made available to users to allow them to experiment and raise problems that they discover with the system developers. This is done at the USER's environment. 3. Acceptance Testing — Customers test a system to decide whether or not it is ready to be accepted from the developers and deployed in the customer environment. This is primarily for custom systems and is the final stage before deployment. Why User Testing is Essential: User testing is essential even after comprehensive system and release testing because influences from the user's working environment have a major effect on the reliability, performance, usability, and robustness of a system. These influences CANNOT be replicated in a testing environment. Only real users in their real environment can uncover these issues.

Q: What is component interface testing? Describe the types of interfaces and common interface errors. [5 marks]

Ans: Component Interface Testing: Software components are composite components made up of several interacting objects. The objective of interface testing is to detect faults due to interface errors or invalid assumptions about interfaces. Testing focuses on showing that the component interface behaves according to its specification. Types of Interfaces: 1. Parameter Interfaces — data passed from one method or procedure to another. 2. Shared Memory Interfaces — a block of memory is shared between procedures or functions. 3. Procedural Interfaces — a sub-system encapsulates a set of procedures to be called by other sub-systems. 4. Message Passing Interfaces — sub-systems request services from other sub-systems. Types of Interface Errors: 1. Interface Misuse — a calling component makes an error in using the interface (e.g., wrong parameter order). 2. Interface Misunderstanding — a calling component embeds incorrect assumptions about the called component's behavior. 3. Timing Errors — called and calling components operate at different speeds; out-of-date information is accessed. Guidelines: Test parameters at extreme ends of ranges; test pointer parameters with null pointers; use stress testing in message passing systems; vary activation order in shared memory systems.

Q: Explain system testing. How does it differ from component testing? What are testing policies? [5 marks]

Ans: System Testing: System testing involves integrating components to create a version of the system and testing the integrated system. The focus is on testing INTERACTIONS between components. System testing checks that components are compatible, interact correctly, and transfer the right data at the right time across their interfaces. It tests the EMERGENT BEHAVIOUR of a system (behaviors that arise only when components work together). Difference from Component Testing: - Component testing tests individual composite components in isolation, focusing on component interfaces. - System testing tests the entire integrated system, focusing on component interactions and emergent behavior. - System testing may include reusable and off-the-shelf components integrated with new ones. - System testing is a COLLECTIVE process; component testing can be more individual. - Some companies use a separate testing team (no programmers/designers) for system testing. Testing Policies (because exhaustive testing is impossible): 1. All system functions accessed through menus should be tested. 2. Combinations of functions accessed through the same menu must be tested. 3. Where user input is provided, all functions must be tested with both correct AND incorrect input.

PART D — QUICK REVISION CHEAT SHEET

Term	One-line Definition
Software Testing	Executing a program with artificial data to find defects and validate behavior.
Validation Testing	Testing to show software meets its requirements; success = system works correctly.
Defect Testing	Testing to expose faults; success = system performs incorrectly (defect found).
Unit Testing	Testing individual components in isolation; defect testing process.
Component Testing	Testing composite components; focus on component interfaces.
System Testing	Testing integrated system; focus on component interactions and emergent behavior.
Release Testing	Black-box testing by separate team before release; focus on validation.
User Testing	Users test system in their own environment (alpha / beta / acceptance).
Alpha Testing	Users + dev team test software at developer's site.
Beta Testing	Release available to users to find problems; done at user's site.
Acceptance Testing	Customer tests system to decide if it is ready for deployment (custom systems).
TDD	Tests written BEFORE code; passing tests drives development; incremental.
Regression Testing	Re-running all tests after changes to ensure nothing is broken.
Partition Testing	Group inputs into equivalence partitions; choose tests from each partition.
Guideline-based Testing	Use guidelines based on common programmer errors to select test cases.
Interface Misuse	Calling component makes error in using interface (e.g., wrong parameter order).
Interface Misunderstanding	Calling component has incorrect assumptions about called component's behavior.
Timing Errors	Called and calling components operate at different speeds; stale data accessed.
Emergent Behavior	Behavior that appears only when components are integrated and work together.
Exhaustive Testing	Testing all possible inputs — IMPOSSIBLE in practice; policies are used instead.

✓ **Good luck in your exam! You have everything you need in these notes.**